이 페이지

1. DP란
2. 1차원 DP
3. 트리 위의 DP: 독립 집합 (independent set)
4. 2차원 DP
   a. Edit distance
   b. Subset sum
   c. Optimal BST
   d. 행렬 연쇄 곱셈
   e. Knapsack
   f. 가중 구간 스케줄링
   g. 구간별 최소 제곱
5. 연습문제

동적 계획법 (dynamic programming)

2026-10-20 (화) · 슬라이드 Dynamic Programming (p.1–28, 녹취 없음, 교수 marked PDF)

DP란

- **이름 유래** (Bellman):
  - 원문 역사 메모: 이름은 Bellman이 붙였고 방법 자체는 12세기부터 있었음
  - 주석: 'Dynamic programming'이라는 용어는 Richard Bellman이 1950년대에 정립한 것으로, 이 용어의 등장 시점과 유사한 아이디어 자체의 역사는 구분해야 한다. 동적 계획법과 유사한 아이디어는 Bellman의 작업보다 훨씬 오래전부터 존재했지만, 이를 현대적 의미의 동적 계획법과 동일한 방법으로 간주할 때는 주의가 필요하다.
  - dynamic = multistage, time-varying process. 문제 구조는 시간에 따라 변하지 않음 → 사실 **static**한 방법 — DP는 부분문제의 관계를 점화식으로 표현하고, 필요한 부분문제의 답을 저장해 재사용하는 방법이다. 최적화 문제에서는 최적 부분 구조가 중요하며, 겹치는 부분문제가 존재하면 중복 계산을 크게 줄일 수 있다. 'dynamic'이라는 이름이 모든 DP 문제에서 상태가 시간에 따라 변해야 한다는 뜻은 아니다

  - programming = 코딩이 아니라 **planning/scheduling**
    - 연주회 곡 순서표(Brahms → Handel → Mozart)와 같은 뜻

DP의 두 단계

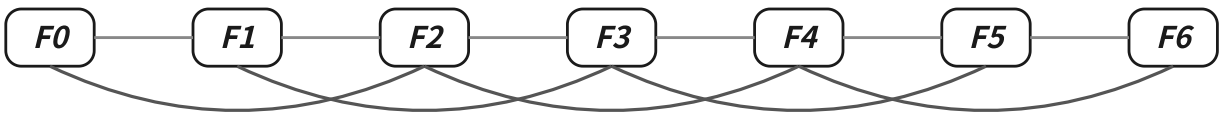
- **1단계** — 문제를 **recursive하게 정의** (점화식 = 부분문제 사이의 relation)
- **2단계** — 점화식을 **순서대로 풀고 해를 저장** (table)

master theorem은 크기 n/b 부분문제 a개의 **균등 분할**만 다룸. DP는 반반·삼등분 같은 고정 분할이 아니라 **문제 성질에서 직접 정의하는 smart recurrence** — 고전적인 Master Theorem은 일정한 크기 비율로 분할되는 특정 형태의 재귀식 분석에 사용된다. DP는 분할 비율이 아니라 부분문제를 정의하고, 그 답을 저장·재사용하는 방식으로 구분한다. 따라서 DP의 점화식은 문제에 따라 다양한 형태를 가질 수 있다

- 잘하려면 부분문제 정의 연습을 많이 하는 수밖에 없음
  - "DP is not about filling in tables, it's about smart recursion"

복습: Fibonacci가 DP의 출발점

- 순진 재귀: $T(n) = T(n-1) + T(n-2) + O(1) \to T(n) = \Theta(\varphi^n), \quad \varphi = \frac{1+\sqrt{5}}{2}$ - 같은 F_k를 매번 처음부터 다시 계산 (호출 수 $C(n) = 1 + C(n-1) + C(n-2) = 2F_{n+1} - 1$)
- table 저장: F_0, F_1 base case부터 $F_2, F_3, \ldots$ 작은 index 순으로 채움 → $T(n) = O(n)$
- 저장하지 않으면 다시 계산해야 함 → **table이 필요한 이유**



위쪽: i−1 → i · 아래쪽: i−2 → i — 한 방향만 있어 사이클 없음 (DAG)

부분문제의 의존 관계 — **ordering(DAG)**: 부분문제 사이의 순서 · **relation**: 작은 부분문제의 답으로 큰 문제를 푸는 식

- 간선이 한 방향(작은 index → 큰 index)뿐 → 사이클 없음 → 작은 것부터 풀면 필요한 값이 항상 먼저 계산되어 있음

- DAG에서 시작 정점 s로부터 각 정점까지의 최단 거리를 구할 때, d(s) = 0으로 초기화하고 나머지 정점의 거리는 ∞로 초기화한다. 정점들을 위상 순서로 처리하면서 각 간선 (u, v)에 대해 d(v) ← min{d(v), d(u) + w(u, v)}로 갱신한다. 동치로, 정점 v의 최단 거리는 d(v) = min_{(u, v)∈E} {d(u) + w(u, v)}로 표현할 수 있으며, 도달할 수 없는 정점의 거리는 ∞로 둔다. 위상 순서에서는 모든 선행 정점이 먼저 처리되므로 각 정점의 최단 거리를 한 번의 순서대로 처리하는 과정에서 계산할 수 있다. 모든 정점과 간선을 처리하는 시간복잡도는 O(|V| + |E|)이다.

항목	top-down memoization	bottom-up table
형태	재귀 함수 + 결과 저장	반복문으로 작은 것부터 채움
푸는 부분문제	호출된 것만	전부
순서	호출이 알아서 결정	직접 정해야 함 (DAG 순서)
이 강의에서	재귀(backtracking)에 저장을 붙인 형태	FIB2, BitString 등 반복문 알고리즘

Top-down memoization은 재귀 호출로 도달한 부분문제를 계산하고 결과를 저장한다. Bottom-up은 의존 관계에 맞는 순서를 정해 작은 부분문제부터 계산하며, 일반적인 구현에서는 전체 표를 채우기도 한다. 실제로 계산하는 상태의 수는 구현 방식과 문제 구조에 따라 달라질 수 있다.

1차원 DP

Bit strings — 1이 연속으로 두 번 나오지 않는 길이 n 문자열 수

- $n = 1$: 0, 1 → 2개 · $n = 2$: 00, 01, 10 → 3개 (11 제외) · $n = 3$: 000, 001, 010, 100, 101 → 5개
- $S(i)$ = 길이 i 유효 문자열 수, $S(1) = 2$, $S(2) = 3$
- **경우 나누기** (첫 글자 기준):
  - **0**으로 시작 → 나머지 $i - 1$칸은 아무 유효 문자열 → $S(i - 1)$
  - **1**로 시작 → 다음은 반드시 0 (**10**) → 나머지 $i - 2$칸 → $S(i - 2)$
  - 길이 n ≥ 2인 유효 문자열은 0으로 시작하거나 10으로 시작한다. 길이 0과 1은 기저 조건으로 처리하며 S(0) = 1, S(1) = 2로 정의한다.

$$S(i) = S(i - 1) + S(i - 2) \quad \Rightarrow \quad S(i) = F_{i+2}$$

- 구현: $T[0] = 1, T[1] = 2$부터 차례로 채워 $T[n]$ 반환 — $O(n)$
- 배열 T는 위의 S(i)를 그대로 저장한 것이며, T[0] = 1은 S(0) = 1(빈 문자열 1개)에 해당한다.

최대 연속 부분 배열 합 (largest sum contiguous subarray)

- 문제: 수가 n개인 배열 A에서 **연속 구간 합의 최댓값** — 예: 회사 손익에서 가장 많이 번 연속 기간
- 슬라이드 배열 `-2 -3 4 -1 -2 1 5 -3` → 답 **7** (3~7번째: 4, −1, −2, 1, 5)
- 점진적 개선: 구간 (i, j)가 $O(n^2)$개, 구간마다 합을 새로 구하면 $O(n^3)$ → 누적합 저장 $O(n^2)$ → **DP** $O(n)$

> $S(i)$ = $A[i]$**에서 끝나는** 연속 구간의 최대 합
>
> $$S(1) = A[1], \qquad S(i) = \max\{\, S(i-1) + A[i],\ \ A[i] \,\}$$
>
> - 끝 위치를 고정하면 선택은 둘뿐: 앞 구간을 **이어 붙이거나** vs **버리고 새로 시작**
> - $S(i-1) > 0$이면 이어 붙임, $S(i-1) \le 0$이면 $A[i]$ 하나로 새로 시작
> - 비유: 물려받은 유산이 양수면 가져가고, 음수면 잊고 새로 시작

- 답은 $\max_i S(i)$ · 시작 index 역추적: 끝에서 거꾸로 가다 $S(i-1) \le 0$인 곳(새로 시작한 지점)
- 시간 $O(n)$, 공간 $O(n)$

손계산 — 배열 −2 −3 4 −1 −2 1 5 −3

i	A[i]	S(i−1)+A[i]	A[i]	S(i)	선택
1	−2	–	−2	−2	base
2	−3	−5	−3	−3	restart
3	4	1	4	4	restart
4	−1	3	−1	3	extend
5	−2	1	−2	1	extend
6	1	2	1	2	extend

7	5	7	5	7	extend
8	−3	4	−3	4	extend

- $\max S(i) = S(7) = 7$
- 구간 역추적: $S(7) = 7 \leftarrow S(6) = 2 \leftarrow S(5) = 1 \leftarrow S(4) = 3 \leftarrow S(3) = 4$, 그 앞 $S(2) = -3 \leq 0$ → 구간은 3~7
- 주의: 마지막 칸 $S(8) = 4$가 답이 아님 — **반드시 모든 i의 max를 봐야 함**

최장 증가 부분수열 (LIS)

- 문제: 수열 $a_1, \ldots, a_n$에서 **순서를 유지하며 값이 증가하는** 가장 긴 부분수열 (연속 아님)
- 슬라이드 수열 `5 2 8 6 3 5 6 9 7 8` → 길이 **6** (2 3 5 6 7 8)
- brute force 2^n개 부분수열 → DP로



LIS 길이 6: 2 3 5 6 7 8 (빨간 동그라미)

$LIS(i)$ = $A[i]$**로 끝나는** LIS 길이, $LIS(1) = 1$

$$LIS(i) = 1 + \max\{\, LIS(j) \mid j < i \ \text{ and } \ A[j] < A[i] \,\}$$

- 바로 앞 원소 조건 둘: **위치** $j < i$ + **값** $A[j] < A[i]$
- 후보 중 $LIS(j)$가 가장 큰 것 뒤에 $A[i]$를 붙임 · 후보가 없으면 $LIS(i) = 1$

- 각 i마다 모든 $j < i$ 검사 → 시간 $O(n^2)$, 공간 $O(n)$
- 답은 $\max_i LIS(i)$ · 수열 역추적: 앞 원소 위치 j를 저장해 두고 거슬러 올라감

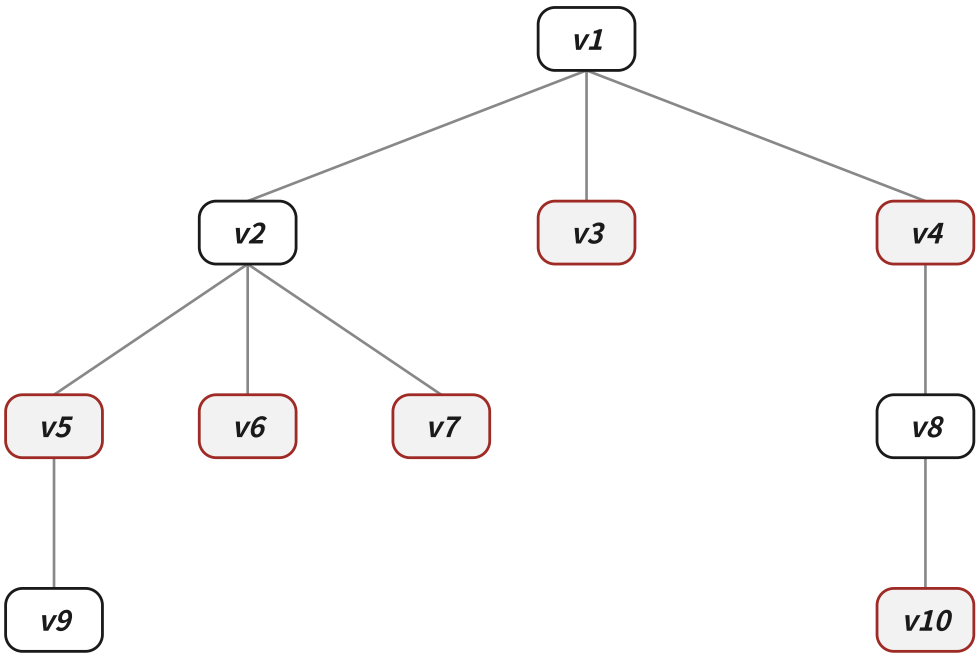
손계산 — 5 2 8 6 3 5 6 9 7 8

i	1	2	3	4	5	6	7	8	9	10
A[i]	5	2	8	6	3	5	6	9	7	8
LIS(i)	1	1	2	2	2	3	4	5	5	6
앞 원소 j	–	–	1	1	2	5	6	7	7	9

- 예: $i = 6$ (값 5): $j < 6$이고 값 < 5인 후보는 $j = 2(2), j = 5(3) \rightarrow 1 + \max(1, 2) = 3$, 앞 원소 $j = 5$
- $i = 10$ (값 8): 후보 $j = 1, 2, 4, 5, 6, 7, 9$ 중 최대는 $LIS(9) = 5 \rightarrow LIS(10) = 6$
- 역추적: 10 ← 9 ← 7 ← 6 ← 5 ← 2 → 값으로 2 3 5 6 7 8

트리 위의 DP: 독립 집합 (independent set)

- 문제: 이웃하지 않은 정점들의 최대 크기 집합. 일반 그래프의 최대 독립 집합 문제는 NP-hard이다. 완전 탐색은 2^n개 부분집합을 후보로 검사한다. 최대 독립 집합을 항상 최적으로 구하는 다항 시간 알고리즘이 존재한다면 P = NP가 성립한다.
- **트리에서는 DP로 $O(n)$**



빨간 노드 = 최대 독립 집합 하나 (크기 6): v3, v4, v5, v6, v7, v10

$IS(u)$ = u의 subtree 안 최대 독립 집합 크기

$$IS(u) = \max\left\{ \sum_{v \in \text{children}(u)} IS(v),\ \ 1 + \sum_{w \in \text{grandchildren}(u)} IS(w) \right\}$$

- u를 **뽑지 않음** → 자식 subtree들은 서로 독립 → 자식의 IS 합
- u를 **뽑음** → 자식은 못 뽑음 → 손자 subtree들이 독립 → $1+$ 손자의 IS 합
- u가 leaf이면 $IS(u) = 1$

최대 독립 집합의 크기를 계산한 뒤 집합 자체를 복원하려면, 각 정점 u에서 A = Σ_{u′∈children(u)} IS(u′)와 B = 1 + Σ_{w∈grandchildren(u)} IS(w)를 비교한다. A ≥ B이면 u를 선택하지 않고 각 자식의 subtree에서 최적 집합을 복원한다. B > A이면 u를 선택하고 모든 자식을 선택하지 않은 뒤, 각 손자의 subtree에서 최적 집합을 복원한다. 두 값이 같으면 어느 쪽을 선택해도 최적해를 얻는다. 각 정점의 선택 여부를 저장하면 O(n) 시간과 O(n) 공간으로 최대 독립 집합 하나를 복원할 수 있다.

손계산 — 슬라이드 트리 (10개 정점)

v1 = root, 자식 v2 v3 v4 · v2의 자식 v5 v6 v7 · v4의 자식 v8 · v5의 자식 v9 · v8의 자식 v10

정점	자식	자식 IS 합	1 + 손자 IS 합	IS(u)
v6, v7, v9, v10, v3	잎	–	–	1
v5	v9	1	1	max(1, 1) = 1
v8	v10	1	1	max(1, 1) = 1
v2	v5, v6, v7	3	2	max(3, 2) = 3
v4	v8	1	2	max(1, 2) = 2
v1	v2, v3, v4	6	5	max(6, 5) = 6

답: $IS(v_1) = 6$ — v4는 뽑는 쪽($1 + IS(v_{10}) = 2$)이 이김에 주의

2차원 DP

Edit distance

- 문제: 두 문자열 사이 최소 편집 횟수 — 편집 거리 0은 두 문자열이 동일함, 1은 삽입·삭제·치환 중 한 번의 편집으로 변환 가능함을 뜻한다. 편집 거리는 표기상 차이를 측정하며 의미 유사성을 단정하지 않는다 (예: 검색어 교정 — 입력과 사전 단어의 거리가 최소인 것)
- 예: SNOWY → SUNNY는 **3**번 (insert U, substitute O→N, delete W). 5번짜리(insert S, S→U, delete O, delete W, insert N)보다 좋은 게 최소
- **alignment**로 읽기: 두 문자열을 나란히 놓고
  - X 쪽 gap = X에 글자 **insert** · Y 쪽 gap = X에서 글자 **delete** · 서로 다른 두 글자 한 열 = **substitute**

$E(i, j)$ = $X[1..i]$와 $Y[1..j]$의 편집 거리

$$E(i,j) = \min \begin{cases} E(i-1,j)+1 & \text{delete} \\ E(i,j-1)+1 & \text{insert} \\ E(i-1,j-1)+\text{diff}(i,j) & \text{substitute?} \end{cases}$$

- $\text{diff}(i,j) = 0$ if $X[i] = Y[j]$, else 1
- base: $E(i,0) = i$ (전부 delete), $E(0,j) = j$ (전부 insert)

- (i,j) 칸은 왼쪽·위·왼위 대각선 세 칸만 봄 → 행 순서로 채우면 됨
- 시간·공간 $O(mn)$ · 경로 역추적은 $E[m,n]$에서 거슬러 $O(m+n)$

손계산 — SNOWY → SUNNY

	∅	S	U	N	N	Y
∅	0	1	2	3	4	5
S	1	0	1	2	3	4
N	2	1	1	1	2	3
O	3	2	2	2	2	3

W	4	3	3	3	3	3
Y	5	4	4	4	4	**3**

$E[5,5] = 3$ — alignment: Y=Y 통과, W→N 치환, O→N 치환, N→U 치환 대신 insert U + substitute O→N + delete W 조합 등 최소 3회

슬라이드 큰 예 — EXPONENTIAL → POLYNOMIAL, $E[11,10] = 6$

EXPONENTIAL의 길이는 11, POLYNOMIAL의 길이는 10이다. E(i,j)를 빈 문자열을 포함한 접두사 간 편집 거리로 정의하면 DP 표는 12×11이다. 최종 편집 거리는 6이다.

	∅	**P**	**O**	**L**	**Y**	**N**	**O**	**M**	**I**	**A**	**L**
∅	0	1	2	3	4	5	6	7	8	9	10
E	1	1	2	3	4	5	6	7	8	9	10
X	2	2	2	3	4	5	6	7	8	9	10
P	3	2	3	3	4	5	6	7	8	9	10
O	4	3	2	3	4	5	5	6	7	8	9
N	5	4	3	3	4	4	5	6	7	8	9
E	6	5	4	4	4	5	5	6	7	8	9
N	7	6	5	5	5	4	5	6	7	8	9
T	8	7	6	6	6	5	5	6	7	8	9
I	9	8	7	7	7	6	6	6	6	7	8
A	10	9	8	8	8	7	7	7	7	6	7
L	11	10	9	8	9	8	8	8	8	7	**6**

- alignment: X 줄 `EXPONENT-IAL` / Y 줄 `--POLYNOMIAL`
- E, X delete 2 + N→L, E→Y, T→O 치환 3 + insert M 1 = **6**

- 참고: 앞부분 EXPONE → POLYNO만 보면 치환 5번(E→P, X→O, P→L, O→Y, E→O)으로 가능 — 하지만 전체 문자열의 최적은 위 조합인 **6**회
- 원본 슬라이드(11/28) 표에서는 i=6 행(E, "EXPONE")의 j=6~10 값이 6,7,8,9,10으로 표기되어 있으나 슬라이드의 재귀식으로 계산하면 5,6,7,8,9가 맞음(E(5,5)=4, diff(E,O)=1 → E(6,6)=5). 위 표는 재귀식대로 표기 — 최종 답 E(11,10)=6에는 영향 없음

Subset sum

- 문제: 수 n개 X(양의 정수) 와 목표 T — 합이 정확히 T인 부분집합이 있나?
- 재귀 먼저: $SS(i, T)$ = $X[1..i]$ 중 합 T 가능? — $X[i]$를 **쓴다/안 쓴다** 두 호출의 OR
- base: $T = 0$ → True · $T < 0$ 또는 $i = 0$ → False. 시간 $O(2^n)$, 공간 $O(n)$ (재귀 깊이)

$$SS(i,t) = \begin{cases} \text{True} & t = 0 \\ \text{False} & i < 1 \\ SS(i-1,t) & t < X[i] \\ SS(i-1,t) \ \vee \ SS(i-1,t-X[i]) & \text{otherwise} \end{cases}$$

- (i, t) 쌍을 표 $S[i, t]$로 저장: $i = 1..n, t = 1..T$ → 시간·공간 $O(nT)$
- O(nT)는 목표값 T의 크기에 비례하는 의사다항(pseudo-polynomial) 시간이다 — 입력의 비트 수 기준으로는 다항 시간으로 볼 수 없다.

손계산 — $X = \{2, 3, 7, 8\}, T = 11$ (원 슬라이드에 수치가 없어 노트가 구성한 예)

i \ t	0	1	2	3	4	5	6	7	8	9	10	11
0 (없음)	T	F	F	F	F	F	F	F	F	F	F	F
1 (2)	T	F	T	F	F	F	F	F	F	F	F	F
2 (3)	T	F	T	T	F	T	F	F	F	F	F	F
3 (7)	T	F	T	T	F	T	F	T	F	T	T	F
4 (8)	T	F	T	T	F	T	F	T	T	T	T	**T**

- $S[4, 11]$ =**True** (실제로 3 + 8 = 11)
- 역추적: $S[3, 11]$이 False이므로 8을 선택해 $t = 3$. $S[2, 3]$이 True이므로 7은 건너뛴다. $S[1, 3]$이 False이므로 3을 선택하면 $t = 0$.
- $T = 6$ 같은 값이면 마지막 행의 해당 칸이 F

Optimal BST

- 입력: key $v_1 < \cdots < v_n$과 검색 빈도 $f[1..n]$ — 목표: **총 검색 비용이 최소인 BST**
- 이 절의 최적 BST 모델은 키가 실제로 존재하는 경우의 성공 검색 빈도만을 고려한다. 존재하지 않는 키에 대한 실패 검색까지 고려하려면 실패 검색 빈도를 별도로 도입해야 하며, 점화식도 이에 맞게 달라져야 한다.

$$\mathrm{Cost}(T, f) = \sum_{i=1}^{n} f[i] \cdot \mathrm{depth}(T, v_i)$$

- 자주 찾는 key를 얕게 두면 검색 비용을 줄일 수 있지만, BST의 key 순서를 지켜야 하므로 빈도만으로 깊이를 정할 수는 없다.
- 루트를 v_r로 정하면 왼쪽 $v_1..v_{r-1}$, 오른쪽 $v_{r+1}..v_n$ — 각 subtree도 최적이어야 함 (**optimal substructure**)
- subtree의 모든 key는 depth가 1씩 커짐 → **그 구간 빈도 합 $F(i, j)$이 한 번씩 더해짐**

$$OPT(i, j) = \sum_{k=i}^{j} f[k] + \min_{i \le r \le j} \{\, OPT(i, r-1) + OPT(r+1, j) \,\}, \qquad OPT(i, j) = 0 \ (i > j)$$

$F(i, j) = F(i, j-1) + f[j]$로 구간 합도 DP로 미리 계산

- 칸 $O(n^2)$개 × 칸마다 $r = i..j$ 시도 $O(n)$ → 시간 $O(n^3)$, 공간 $O(n^2)$
- **채우는 순서 세 가지** (모두 $(i, r-1)$, $(r+1, j)$가 먼저 계산되게):
  - ① 대각선 단위: 바깥 $d = 0..n-1$, 안쪽 $i = 1..n-d$ → 칸 $(i, i+d)$
  - ② 아래 행부터: 바깥 $i = n+1 \to 1$, 안쪽 $j = i..n$

- ③ 왼쪽 열부터: 바깥 $j = 1..n$, 안쪽 $i = j \to 1$ (열 안은 아래→위)

손계산 — 빈도 $f = (4, 2, 6, 3)$

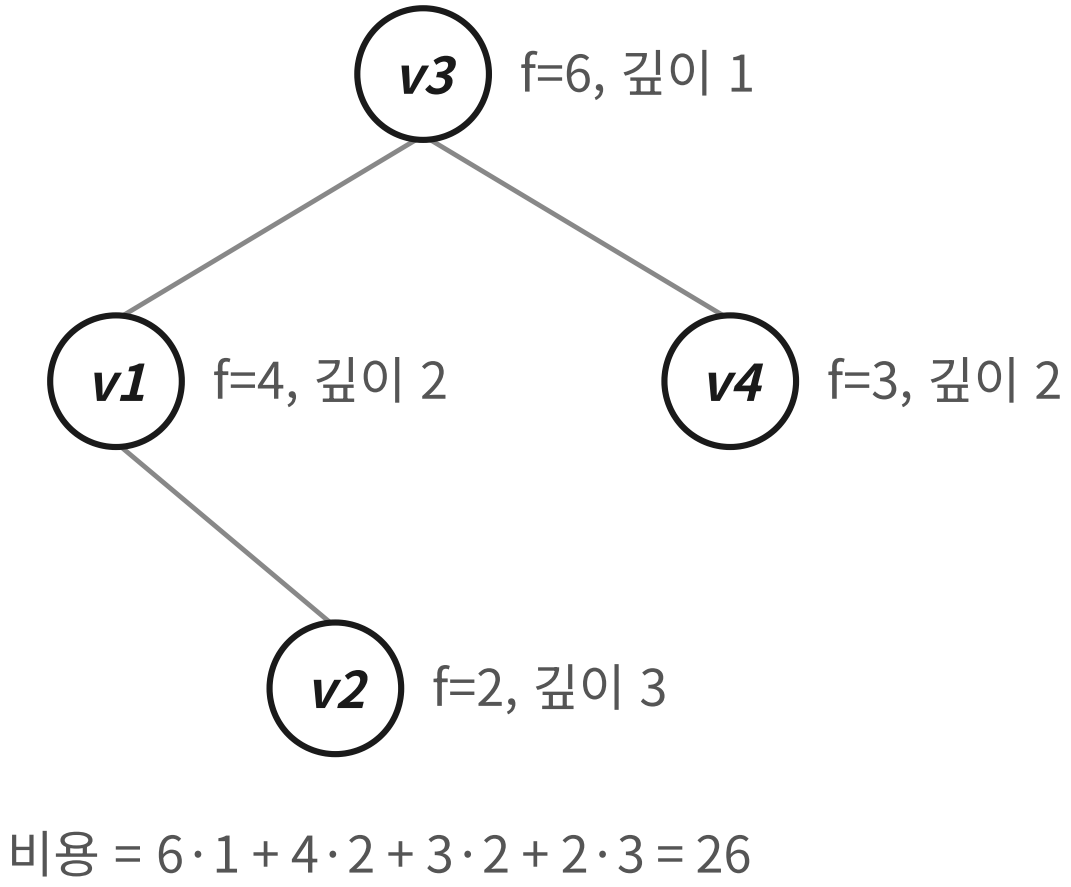
- 구간 합: $F(1,2) = 6$, $F(1,3) = 12$, $F(1,4) = 15$, $F(2,4) = 11$, $F(3,4) = 9$

OPT(i, j)	j = 1	j = 2	j = 3	j = 4
i = 1	4 (r = 1)	8 (r = 1)	20 (r = 3)	**26 (r = 3)**
i = 2	–	2 (r = 2)	10 (r = 3)	16 (r = 3)
i = 3	–	–	6 (r = 3)	12 (r = 3)
i = 4	–	–	–	3 (r = 4)

$OPT(1,4)$ 후보 비교 (합 + $F(1,4) = 15$):

루트 r	OPT(1, r−1)	OPT(r+1, 4)	합	+ 15
1	0	16	16	31
2	4	12	16	31
3	8	3	11	**26**
4	20	0	20	35

루트 v_3 선택 → 왼쪽 $(1,2)$의 루트 v_1, 오른쪽 $(4,4)$ = v_4 → 비용 검산:
$6 \cdot 1 + 4 \cdot 2 + 3 \cdot 2 + 2 \cdot 3 = 26$ ✓ (아래 그림)



최적 BST — 루트 v_3, 왼쪽 v_1-v_2, 오른쪽 v_4

행렬 연쇄 곱셈 (chain matrix multiplication)

- 문제: $A_1 \times \cdots \times A_n$ (A_i는 $m_{i-1} \times m_i$) — **괄호를 어디에 치느냐**에 따라 곱셈 횟수가 크게 달라짐
- $m \times n$ 행렬과 $n \times \ell$ 행렬의 곱 = $mn\ell$**번**의 스칼라 곱
- 같은 4개 행렬이라도 ($m = (50, 20, 1, 10, 100)$):

괄호 치는 법	곱셈 횟수
((A × B) × C) × D	51,500
A × ((B × C) × D)	120,200
(A × (B × C)) × D	60,200
(A × B) × (C × D)	**7,000**

A × (B × (C × D))	103,000

$\mathrm{Cost}(i,j)$ = $A_i \times \cdots \times A_j$의 최소 곱셈 횟수 — 마지막 곱셈 위치 k를 기준으로 쪼갬

$$\mathrm{Cost}(i,j) = \min_{i \le k < j} \{ \mathrm{Cost}(i,k) + \mathrm{Cost}(k+1,j) + m_{i-1} \cdot m_k \cdot m_j \}, \quad \mathrm{Cost}(i,i) = 0$$

마지막 곱셈은 $(m_{i-1} \times m_k)$ 행렬과 $(m_k \times m_j)$ 행렬의 곱 → $m_{i-1} \cdot m_k \cdot m_j$

- 채우기: 길이 $\ell = 1..n-1$ 순서로, 각 i에 $j = i + \ell$ (optimal BST와 같은 대각선 순서)
- 시간 $O(n^3)$, 공간 $O(n^2)$

손계산 — $m = (50, 20, 1, 10, 100)$

C[i, j]	j = 1	j = 2	j = 3	j = 4
i = 1	0	1,000 (k = 1)	1,500 (k = 2)	**7,000 (k = 2)**
i = 2	–	0	200 (k = 2)	3,000 (k = 2)
i = 3	–	–	0	1,000 (k = 3)
i = 4	–	–	–	0

- $C[1,3]$: $k=1$이면 $0 + 200 + 50 \cdot 20 \cdot 10 = 10{,}200$, $k=2$이면 $1000 + 0 + 50 \cdot 1 \cdot 10 = 1{,}500$ → 1,500
- $C[1,4]$: $k=1 \to 0 + 3000 + 100000 = 103{,}000$ · $k=2 \to$ $1000 + 1000 + 5000 = 7{,}000$ · $k=3 \to 1500 + 0 + 50000 = 51{,}500$ → **7,000**
- 역추적: $k=2$에서 쪼갬 → **(A × B) × (C × D)**

Knapsack

- 문제: 용량 W, 물건 n개 (무게 w_i, 가치 v_i) — 가치 합 최대
- 슬라이드 예 $W = 10$: 무제한이면 item 1 + item 4 둘 = 29 + 18 = 47 · 하나씩만이면 item 1 + item 3 = 45

무제한 (unbounded)

$K(w)$ = 용량 w의 최대 가치

$$K(0) = 0,\ K(w) = \max_{1 \le i \le n,\ w_i \le w} \{\, K(w - w_i) + v_i \,\}$$

마지막에 넣을 물건 i를 고름 — 같은 물건을 여러 번 사용할 수 있다. 넣을 수 있는 물건이 없으면 $K(w) = 0$.

하나씩만 (0/1)

$K(w, j)$ = 용량 w, 물건 $1..j$만 사용

$$K(w, j) = \begin{cases} K(w, j-1) & w_j > w \\ \max\{K(w, j-1),\ K(w - w_j, j-1) + v_j\} & \text{otherwise} \end{cases}$$

j번째를 넣을지 말지 — 넣으면 $j-1$까지만 더 사용

항목	unlimited	0/1
부분문제	$K(w)$	$K(w, j)$
선택	마지막에 넣을 물건	j번째를 넣을지 말지
시간	$O(Wn)$	$O(Wn)$
공간	$O(W + n)$	$O(Wn)$

표 기반 구현에서는 용량 W와 무게 w_i가 정수라고 가정하며 무제한 배낭에서는 w_i > 0이어야 한다. 이 조건에서 시간 복잡도는 O(nW)이며 의사다항 시간이다. 무제한 배낭은 공간 O(W), 0/1 배낭의 전체 표는 공간 O(nW)를 사용한다. 따라서 W가 입력 길이에 비해 매우 클 때는 이 DP가 효율적이지 않을 수 있다.

손계산 — $W = 10$, 물건 (6, 29), (3, 14), (4, 16), (2, 9)

- 무제한: $K[10]$ 후보 $K[4] + 29 = 47$, $K[7] + 14 = 46$, $K[6] + 16 = 45$, $K[8] + 9 = 47$ → **47** ($K[4] = 18$ = v_1 아니라 $2v_4$, 즉 item 4 둘 + item 1)
- 0/1 표 (w 행 × j 열):

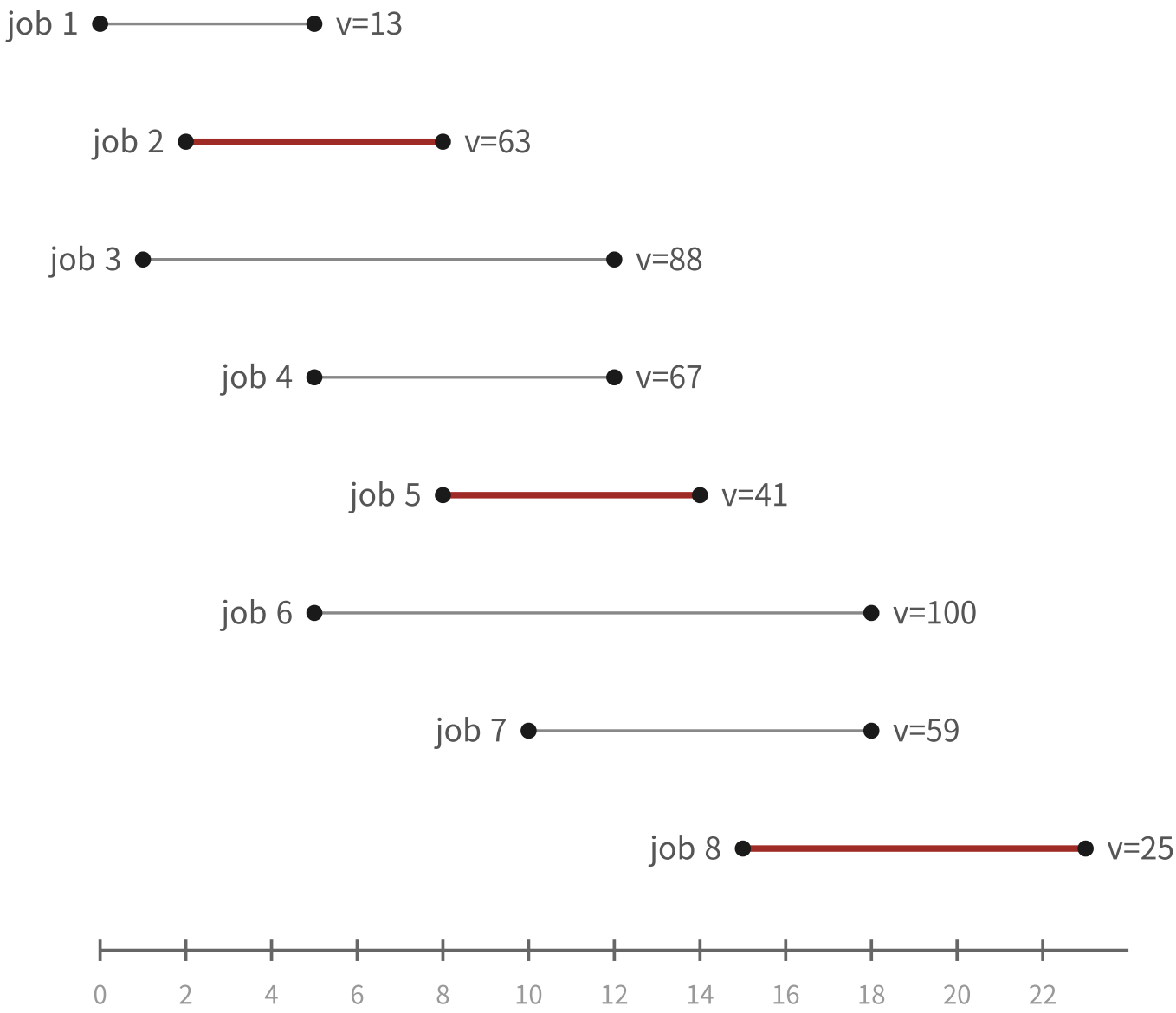
w \ j	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	9
3	0	0	14	14	14
4	0	0	14	16	16
5	0	0	14	16	23
6	0	29	29	29	29
7	0	29	29	30	30
8	0	29	29	30	38
9	0	29	43	43	43
10	0	29	43	45	**45**

- $K[10, 4] = 45 - K[10, 3] = 45$에서 물건 4를 안 넣은 경우와 같음; $K[10, 3]$은 $K[6, 2] + 16 = 29 + 16 = 45$ (item 1 + item 3)

가중 구간 스케줄링 (weighted interval scheduling)

- 문제: job j가 시작 s_j, 끝 f_j, 가치 v_j — 겹치지 않게 골라 가치 합 최대
- 끝나는 시간 순 정렬 $f_1 \le f_2 \le \cdots \le f_n$이 전제
- $p(j)$ = job j와 compatible한 마지막 번호 ($i < j$ 중 $f_i \le s_j$인 최대 i)
  - $p(j)$ 이전 번호는 모두 compatible · $p(j)+1 .. j-1$은 j와 겹침
  - 이분 탐색으로 $p(j)$ 하나에 $O(\log n)$ (s_j를 $f_1 .. f_{j-1}$에서 탐색)

빨강 = 최적 선택 (job 2, 5, 8) → 63+41+25 = 129



$OPT(j)$ = job $1..j$만 있을 때 최대 가치

$$OPT(0) = 0, \qquad OPT(j) = \max\{\, v_j + OPT(p(j)),\ \ OPT(j-1) \,\}$$

- job j **선택** → 겹치는 job 제외 → $v_j + OPT(p(j))$
- job j **미선택** → $OPT(j-1)$

- 정렬 $O(n \log n)$ + 표 채우기 $O(n)$ → 전체 $O(n \log n)$, 공간 $O(n)$

항목	greedy	DP

방식	기준 하나로 정렬해 차례로 고르고 되돌리지 않음	선택/미선택 두 경우를 모두 계산해 큰 쪽
가치가 모두 동일한 양수인 경우	earliest finish가 최적	DP와 같은 최적값
가중치 제각각	기준(시작, 끝, 길이, 충돌 수)마다 실패 가능	**항상 최적**

모든 작업의 가치가 동일한 양수라면, 전체 가치의 최댓값을 구하는 것은 선택한 작업 개수를 최대화하는 것과 같다. 따라서 종료 시각이 가장 빠른 작업부터 선택하는 그리디 알고리즘이 최적이며, DP로 구한 최적값과 일치한다. 작업의 가치가 서로 다르면 작업 개수만 최대화하는 그리디 기준은 전체 가치의 최적성을 보장하지 못하므로, 선택하는 경우와 선택하지 않는 경우를 모두 계산하는 DP를 사용한다.

손계산 — 슬라이드의 8개 job

j	s_j	f_j	v_j	p(j)	v_j + OPT(p(j))	OPT(j−1)	OPT(j)
1	0	5	13	0	13	0	13
2	2	8	63	0	63	13	63
3	1	12	88	0	88	63	88
4	5	12	67	1	80	88	88
5	8	14	41	2	104	88	104
6	5	18	100	1	113	104	113
7	10	18	59	2	122	113	122
8	15	23	25	5	129	122	**129**

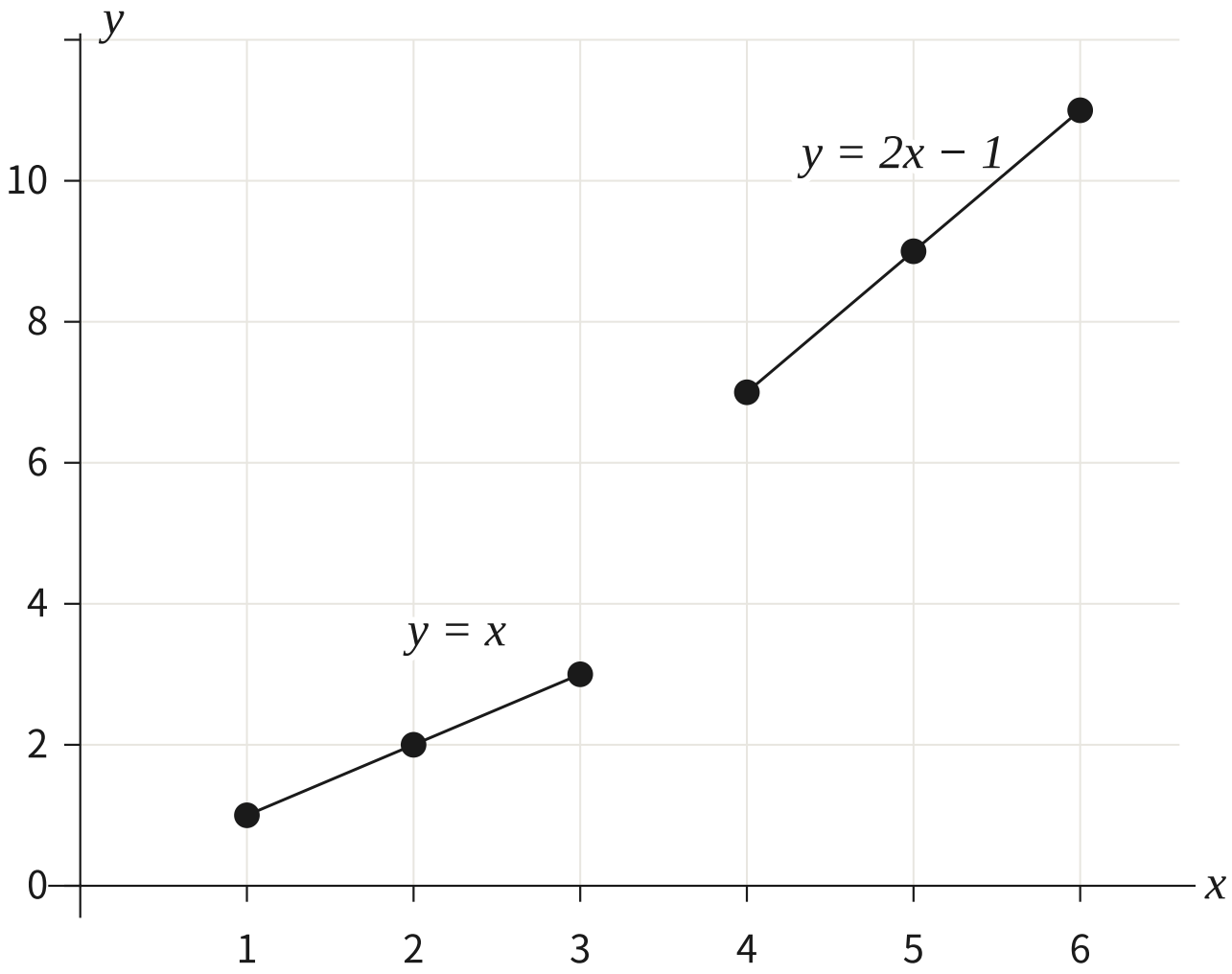
- $OPT(8) = 129$
- 역추적: job 8 선택(129 > 122) → $p(8) = 5$ → job 5 선택(104 > 88) → $p(5) = 2$ → job 2 선택(63 > 13) → $p(2) = 0$ → 선택: **job 8, 5, 2**
- 주의: job 6(가치 100)은 $p(6) = 1$이라 선택해도 113 — 가장 비싼 job 하나가 답은 아님

구간별 최소 제곱 (segmented least squares)

- 문제: 점 n개 $(x_1, y_1), \ldots, (x_n, y_n)$ $(x_1 < \cdots < x_n)$를 **직선 여러 개**로 근사

$$= E + c \cdot L$$

E = 오차 제곱합 $\sum (y_i - ax_i - b)^2$ · L = 직선(구간) 개수 · $c > 0$은 직선 하나당 벌점 — c가 작으면 segment를 많이, 크면 적게 씀



점 (1,1)(2,2)(3,3)(4,7)(5,9)(6,11) — 앞 3개는 $y = x$, 뒤 3개는 $y = 2x - 1$에 정확히 올라탐 (오차 0인 분할이 존재)

$OPT(j)$ = 점 $p_1..p_j$의 최소 비용 · $e(i,j)$ = 점 $p_i..p_j$에 직선 하나를 맞출 때 최소 제곱합

i=j인 단일점의 경우, 직선 회귀 계수 공식의 분모가 0이 될 수 있으므로 별도로 e(i,i)=0으로 정의한다. 단일점은 그 점을 지나는 직선으로 오차 없이 근사할 수 있다.

$$OPT(0) = 0, \qquad OPT(j) = \min_{1 \le i \le j} \{\, e(i,j) + c + OPT(i-1) \,\}$$

마지막 segment가 $p_i..p_j$라면 그 앞 $p_1..p_{i-1}$은 독립 문제

- 직선 계수 (합은 모두 $k = i..j$ 범위):

$$a_{ij} = \frac{(j-i+1)\sum x_k y_k - (\sum x_k)(\sum y_k)}{(j-i+1)\sum x_k^2 - (\sum x_k)^2}, \qquad b_{ij} = \frac{\sum y_k - a_{ij}\sum x_k}{j-i+1}$$

- 직접 계산하면 $e(i,j)$ 하나에 $O(j-i)$ → 전체 $O(n^3)$
- **누적합** $\sum x_k,\ \sum y_k,\ \sum x_k^2,\ \sum x_k y_k,\ \sum y_k^2$을 미리 저장하면 $e(i,j)$가 $O(1)$ → 모든 쌍 $O(n^2)$
- 전체: 표 $M[1..n]$ 채우기 $O(n^2)$ · 역추적: $j = 0$이면 종료, 아니면 $e(i,j) + c + M[i-1]$을 최소로 하는 i를 찾아 $p_i..p_j$ 출력 후 $j \leftarrow i-1$

손계산 — 점 6개,$c = 1$

- 앞 3개는 $y = x$ 위, 뒤 3개는 $y = 2x - 1$ 위 → 두 구간이면 오차 $e = 0$
- $OPT(6)$: $i = 1$이면 한 직선 $3.14 + 1 + 0 = 4.14$ · $i = 4$이면 $0 + 1 + OPT(3) = 2$ → **2**
- 역추적: $j = 6$에서 $i = 4$ → $p_4..p_6$ 출력 · $j = 3$에서 $i = 1$ → $p_1..p_3$ 출력 → 두 segment
- $c = 5$로 벌점이 크면: 두 구간 $0 + 5 + 0 + 5 = 10$ > 한 직선 $3.14 + 5 = 8.14$ → 한 직선 선택 — **c가 구간 수를 조절**

동적 계획법: 최장 공통 부분수열 (LCS)

		A	B	C	B	D	A
	0	0	0	0	0	0	0
B	0						
D	0						
C	0						
A	0						
B	0						
A	0						
	0						

X=ABCBDAB, Y=BDCABA. dp[i][j] = X 앞 i글자와 Y 앞 j글자의 LCS 길이. 0행·0열은 0.

◀ 이전 1 / 44 다음 ▶

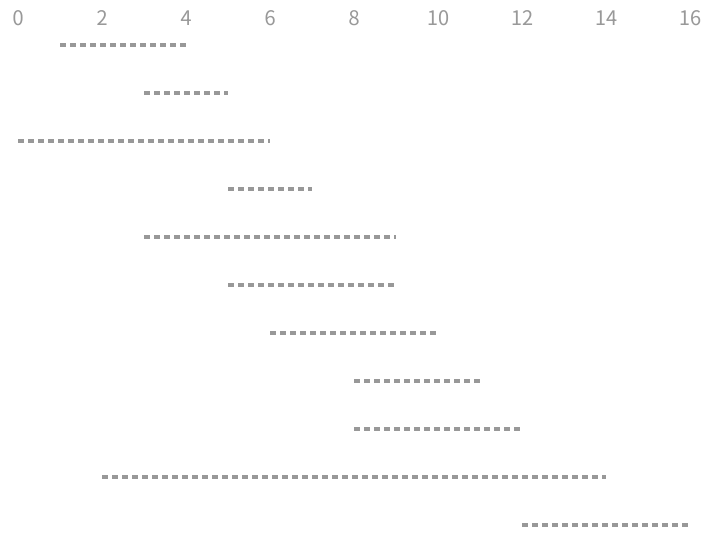
동적 계획법: 0-1 배낭 (Knapsack)

i\w	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0

물건 i(무게 w, 가치 v): (1,1) (3,4) (4,5) (5,7). 배낭 용량 W=7. dp[i][c] = 앞 i개로 용량 c에서의 최대 가치.

◀ 이전 1 / 30 다음 ▶

그리디: 활동 선택 문제



활동 11개 (시작, 끝). **종료 시각이 빠른 순**으로 정렬해 둔 상태. 그리디 선택: 매번 가장 일찍 끝나는 호환 활동을 고른다.

◀ 이전 1 / 6 다음 ▶

연습문제

문제 1

길이 n bit string 중 **1이 연속 세 번 이상 나오지 않는** 것의 개수 $T(n)$의 점화식을 세우고 $T(5)$를 구하시오.

힌트 보기

bit strings 문제와 같은 기법: **앞부분이 0, 10, 110** 중 무엇인지로 경우를 나눔.

정답 보기

개념은 위 힌트 참고.

① 경우 나누기: 0으로 시작 → 나머지 $T(n-1)$ · 10으로 시작 → $T(n-2)$ · 110으로 시작 → $T(n-3)$ (111은 금지)

② 점화식: $T(n) = T(n-1) + T(n-2) + T(n-3)$ $(n \geq 3)$, base $T(0) = 1$, $T(1) = 2, T(2) = 4$

③ 계산: $T(3) = 4 + 2 + 1 = 7, T(4) = 7 + 4 + 2 = 13, T(5) = 13 + 7 + 4 = 24$

최종 답 $T(n) = T(n-1) + T(n-2) + T(n-3)$, $T(5) = 24$

문제 2

배열 `5 -7 3 4 -2 6 -10 2` 에서 합이 최대인 연속 부분배열의 합과 구간을 구하시오.

힌트 보기

$S(i) = \max\{S(i-1) + A[i],\ A[i]\}$ — 앞 합이 양수면 잇고 아니면 새로 시작.

정답 보기

개념은 위 힌트 참고.

① $S(1) = 5, S(2) = \max(5-7, -7) = -2$

② $S(2) \le 0$이라 restart: $S(3) = \max(-2+3, 3) = 3$

③ 이어 붙이기: $S(4) = 3+4 = 7, S(5) = 7-2 = 5, S(6) = 5+6 = 11$

④ $S(7) = 11-10 = 1, S(8) = \max(1+2, 2) = 3$ → max는 $S(6) = 11$

⑤ 구간 역추적: $S(6) \leftarrow S(5) \leftarrow S(4) \leftarrow S(3), S(2) = -2 \le 0$에서 끊김 → 구간 3~6

최종 답 **합 11, 구간 `3 4 -2 6` (index 3~6)**

문제 3

수열 `3 1 4 1 5 9 2 6`의 LIS 길이와 LIS 하나를 구하시오.

힌트 보기

$LIS(i) = 1 + \max\{LIS(j) : j < i,\ A[j] < A[i]\}$, 후보 없으면 1.

정답 보기

개념은 위 힌트 참고.

① $LIS(1) = 1$ (3), $LIS(2) = 1$ (1), $LIS(3) = 1 + LIS(2) = 2$ (1,4), $LIS(4) = 1$ (1)

② $LIS(5) = 1 + \max(LIS(2), LIS(3), LIS(4)) = 1 + 2 = 3$ (1,4,5)

③ $LIS(6) = 1 + 3 = 4$ (1,4,5,9), $LIS(7) = 1 + 1 = 2$ (1,2),
$LIS(8) = 1 + \max(LIS(2), LIS(3), LIS(4), LIS(5), LIS(7)) = 1 + 3 = 4$ (1,4,5,6)

④ max는 4 (i = 6 또는 8)

최종 답 **길이 4 — LIS 하나: `1 4 5 9` (또는 `1 4 5 6`)**

문제 4

r의 자식 a, b / a의 자식 c, d / b의 자식 e / e의 자식 f, g 인 트리에서 최대 independent set의 크기를 점화식으로 구하시오.

힌트 보기

$IS(u) = \max\{\sum_{\text{child}} IS,\ 1 + \sum_{\text{gchild}} IS\}$ — 잎부터 올라옴.

정답 보기

개념은 위 힌트 참고.

① 잎: $IS(c) = IS(d) = IS(f) = IS(g) = 1$

② $IS(e) = \max(IS(f) + IS(g),\ 1 + 0) = \max(2, 1) = 2$ (e를 안 뽑음)

③ $IS(a) = \max(1 + 1,\ 1 + 0) = 2$,
$IS(b) = \max(IS(e),\ 1 + IS(f) + IS(g)) = \max(2, 3) = 3$ (b를 뽑음)

④ $IS(r) = \max(IS(a) + IS(b),\ 1 + IS(c) + IS(d) + IS(e)) = \max(5,\ 5) = 5$

최종 답 **최대 크기 5 — 예: {b, c, d, f, g} (r 안 뽑음) 또는 {r, c, d, f, g} (r 뽑음)**

문제 5

SUNNY를 SUNDAY로 바꾸는 edit distance를 table로 구하고 alignment를 하나 쓰시오.

힌트 보기

$$E(i,j) = \min\{E(i-1,j)+1,\ E(i,j-1)+1,\ E(i-1,j-1)+\mathrm{diff}(i,j)\}.$$

정답 보기

개념은 위 힌트 참고.

① 초기 행·열: ∅ 줄은 0..6, 첫 열은 0..5

② 표 채우기:

	∅	S	U	N	D	A	Y
∅	0	1	2	3	4	5	6
S	1	0	1	2	3	4	5
U	2	1	0	1	2	3	4
N	3	2	1	0	1	2	3
N	4	3	2	1	1	2	3
Y	5	4	3	2	2	2	**2**

3. **Step 3** — $E[5,6] = 2$. 거슬러 가면: Y=Y (대각) → N→A 치환 (대각) → D는 insert (왼쪽)

최종 답

거리 2 — alignment: S U N - N Y → S U N D A Y, 즉 insert D + substitute N→A

문제 6

물건 4개 (무게, 가치) = (6, 29), (3, 14), (4, 16), (2, 9)에서 $W = 7$일 때 **무제한**과 **0/1** 각각의 최대 가치를 구하시오.

힌트 보기

무제한: $K(w) = \max_i \{K(w - w_i) + v_i\}$ · 0/1:
$K(w, j) = \max\{K(w, j-1),\ K(w - w_j, j-1) + v_j\}$.

정답 보기

개념은 위 힌트 참고.

① 무제한 $K[7]$ 후보: $K[1] + 29 = 29$, $K[4] + 14 = 18 + 14 = 32$,
$K[3] + 16 = 14 + 16 = 30$, $K[5] + 9 = 23 + 9 = 32$

② 최대 32 (예: 물건 2 + 물건 4 + 물건 4 = 3+2+2 무게, 14+9+9 가치)

③ 0/1: 손계산 표의 $w = 7$ 행 →
$K[7, 4] = \max(K[7, 3],\ K[5, 3] + 9) = \max(30, 25) = 30$

최종 답

무제한 32 (물건 2 + 물건 4 × 2 = 무게 7, 가치 14 + 9 + 9) · 0/1 30 (물건 2 + 물건 3 = 무게 7, 가치 14 + 16)